PYTHON FOR VIDEO GAMES WITH PYGAME · LESSON 1

Your first window and the game loop

Goal: By the end you can create a virtual environment, install pygame-ce and open a game window with a loop that can be closed.
Before you start:

- Python basics - variables, if, for and functions (lesson 0 of the course).
- Linux or macOS with Python 3.10 or newer. On Windows, see the note in the summary.
- The course folder downloaded and unzipped (it includes the assets folder).

To copy the code: when you copy from a PDF, many viewers drop the spaces at the start of each line, and in Python those spaces matter. Every step is also a `.py` file in the same folder as this guide: open it and copy from there.

Step by step

Your first window and the game loop

Welcome to Python for video games, where you will build Most Wanted UFO Mini, a space shooter, step by step. In this lesson you set up your project and install pygame. Then you learn the most important idea in any game: the game loop. And finally you open your first window, with the game's background.

Step 2

Open a terminal in the course folder. There you already have a folder called assets, with the game's images and sounds. Type ls assets, and press Enter, to see what is inside. Files ending in .png are images, and files ending in .wav are sounds. There is also a license: these assets come from Most Wanted UFO Classic, and you can use them if you give credit.

Type in the terminal:

```
ls assets
```

Step 3

Before installing anything, create a virtual environment: a separate folder where this project's libraries live. Type python3 -m venv .venv, and press Enter. Now activate it: type source .venv/bin/activate, and press Enter. Notice that (.venv) appears in parentheses at the start of the line: the environment is active.

Type in the terminal:

```
python3 -m venv .venv
source .venv/bin/activate
```

Step 4

With the environment active, install pygame-ce: it is the community edition of pygame, and you use it the same way. Type pip install pygame-ce, and press Enter. pip downloads the library, and installs it inside your environment. When you see Successfully installed, it is ready.

Type in the terminal:

```
pip install pygame-ce
```

Step 5

Now create a file called game.py, in the course folder. The first line imports pygame: that way your program can use everything the library brings, like windows, images and sounds. Then you start it with pygame.init(), which prepares the screen, the keyboard and the sound. With pygame ready, you create the window, eight hundred by six hundred pixels, with set_mode, and give it a title, which shows at the top, in the window's bar. You will use the clock in the loop, so the game runs at the same speed on any computer. And finally you load the background image, from the assets folder. The path starts at the folder where you run the game, so the terminal has to be in the course folder. And convert gets the image ready to be drawn fast.

Your `game.py` now looks like this (file `step-1.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
```

Step 6

Now comes the heart of every game: the loop. You use a variable, running, set to True, and a while loop that repeats as long as it stays that way. On each pass, you first check the events: keys, clicks, or someone closing the window. If the QUIT event arrives, which means closing the window, running becomes False, and the loop ends. Then you draw: blit pastes the background in the top left corner, and flip shows the frame, the new picture. And clock.tick(60) makes all of this happen sixty times per second, no faster and no slower. Notice that the last line has no indentation: it is outside the loop. When the loop ends, pygame.quit() closes the window, and frees everything pygame was using.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-2.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
```

```
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.blit(background, (0, 0))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 7

Save the file, and in the terminal type python game.py. The window opens, with the starry background of Most Wanted UFO Mini. Nothing moves yet, but the loop is already running sixty times per second. To close it, click the X on the window: that sends the QUIT event, and the loop ends.

Try it:

```
python game.py
```

What you learned

Let's review. Each project has its own virtual environment. You installed pygame-ce with pip. The game loop checks events, draws, and repeats. And the QUIT event lets you close the window. Your exercise: change the window title, and draw the UFO on top of the background. In the next lesson, the UFO appears, and moves with the keyboard.

- One virtual environment per project
- pip install pygame-ce
- The loop: events, draw, repeat
- QUIT closes the window

Summary

- `python3 -m venv .venv` creates a virtual environment; `source .venv/bin/activate` activates it. **On Windows:** `.venv\Scripts\activate`.
- If `venv` fails on Ubuntu or Debian: `sudo apt install python3-venv`. On Windows use `py -m venv .venv`.
- `pip install pygame-ce` installs pygame inside the active environment; in code you still write `import pygame`.
- `pygame.display.set_mode((800, 600))` creates the window and `pygame.image.load(...)` loads an image.
- The game loop repeats while `running` is `True`: it checks events, draws with `blit`, shows the frame with `flip` and waits with `clock.tick(60)`.

- The `pygame.QUIT` event arrives when the window is closed: that is where you set `running = False`.

Exercise

Start from this lesson's `game.py`.

- Change the window title to one of your own.
- Load `assets/player_1.png` the way you loaded the background and draw it after the background at `(368, 480)`. What happens if you draw it before the background?
- Bonus: make the window 1024 by 768. What happens to the background, which is 800 by 600?

Solution

```
pygame.display.set_caption("My first game")
ufo = pygame.image.load("assets/player_1.png").convert_alpha()
# inside the loop, after the background:
screen.blit(background, (0, 0))
screen.blit(ufo, (368, 480))
```

If you draw the UFO before the background, the background covers it: each `blit` paints over what came before. With a 1024 by 768 window, the background only covers the top-left corner: the rest stays black.

Art and sounds: Most Wanted UFO Classic, by Jose Daniel Soto, licensed CC BY 4.0.